

PE00 — Expression Evaluation

A program that reads lines of input code, converts the expression part of each line from infix to postfix form, evaluates it, and reports the results, the variables used, and any errors found.

Each input code is either

- an **assignment statement**, `var = expression`, whose result is stored in `var`, or
- a plain **expression**, which is evaluated and reported.

Supported operators: `+`, `-`, `*`, `/`, `%`. Parentheses and a leading sign (`-5`) are also accepted.

Requirements

Python 3 with `tkinter` (bundled with the standard Windows and macOS installers; on Debian/Ubuntu install `python3-tk`). No other dependencies.

Running the program

```
python main.py
```

1. Type input codes into the left text area, or click **Load File** to load a `.in` file from any directory — its contents replace whatever is in the input area.
2. Click **Process**. The button stays disabled while the input area is empty.
3. The output appears in the right text area, replacing whatever was

there.

The window stays open until you close it.

Files

File	Contents
<code>main.py</code>	The graphical user interface, and the program entry point
<code>evaluator.py</code>	Parsing, infix-to-postfix conversion, and evaluation
<code>sample.in</code>	A small input file exercising each kind of error

The two modules meet at a single function, `evaluator.run(lines)`, which takes the input area's contents already split into lines and returns the complete output text.

Modules and functions

`main.py` — user interface

Function	Description
<code>get_input_text()</code>	Returns the input area's contents
<code>set_output(text)</code>	Writes the output area, unlocking and relocking it so it stays read-only
<code>refresh_process_state()</code>	Enables Process only while the input area holds something
<code>load_file()</code>	Opens a <code>.in</code> file and displays it in the input area

```
process()
```

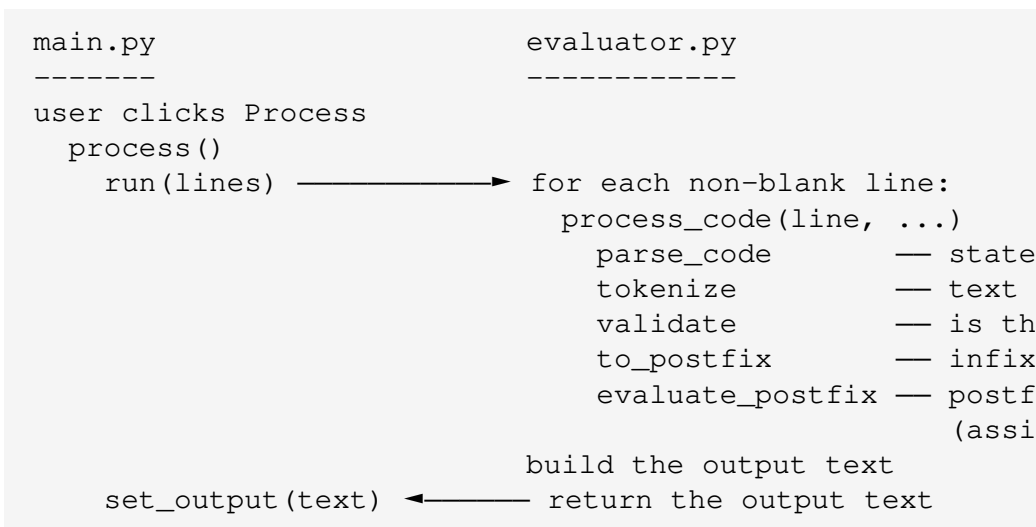
Hands the input lines to the evaluator and displays the result

evaluator.py — processing

Function	Description
<code>run(lines)</code>	Processes every input code and builds the complete output text. The only function the GUI calls
<code>process_code(line, variable_values, variables_used)</code>	Processes one input code end to end and returns its postfix form and its result
<code>parse_code(line)</code>	Splits one code into (target_variable, expression_text); the target is None for a plain expression
<code>tokenize(expression)</code>	Turns the expression text into a list of (kind, value) tokens
<code>validate(tokens)</code>	Checks that the tokens form a well-formed infix expression
<code>to_postfix(infix_tokens)</code>	Converts infix tokens to postfix using the shunting-yard algorithm
<code>evaluate_postfix(postfix_tokens, variable_values)</code>	Evaluates postfix tokens with an operand stack and returns the value

<code>apply_operator(operator, left_operand, right_operand)</code>	Combines two operands with one binary operator
<code>integer_divide / integer_modulo</code>	C-style <code>/</code> and <code>%</code> for whole numbers
<code>is_valid_name(name)</code>	Applies the variable naming rules
<code>format_value / format_postfix</code>	Render a value and a postfix token list for display

Control flow



`variable_values` (name to its most recently assigned value) and `variables_used` (names in order of first appearance) are created in `run` and passed down, so each **Process** click starts from a clean state.

Input format

One code per line; blank lines are skipped.

```
x = 5 + 3
y = x * 2
z = y % 0
a + b
w = 10 / 2
```

A variable name follows the C rules without the underscore: a letter, then any number of letters and digits. A variable's value in an expression is the one most recently assigned to it before that line.

Output format

Three lines per input code, then the variables used, then the errors found, with a blank line between sets.

```
Line: x = 5 + 3
Postfix: 5 3 +
Result: x = 8
```

...

Variables used:

```
x = 8
y = 16
z = undefined
```

Errors found:

```
Line 3: Division by zero
```

A variable that never received a value is listed as `undefined`. When no errors were found, the last set reads `(none)`.

Error handling

Every problem with an input code is raised as a `CodeError` and handled for that line alone, so one bad code never stops the rest of the input from being processed. Each error appears twice: as the `Result :` line for the code, and in the `Errors found :` set at the end with its line number.

Error	Raised when
Invalid input code	The line is not a valid statement or expression: a bad character, a malformed number (1.2.3, 12abc), a bad variable name (1x = 3), a missing or doubled =, unbalanced parentheses, or misplaced operators (3 +, 3 4, 3 * * 4)
Undefined variable <name>	The variable has no value prior to the code using it
Division by zero	The right operand of / or % evaluates to zero

Division by zero in a statement leaves the target variable's previous value untouched, as required: nothing is assigned when the evaluation fails.

Beyond those, the program is written so that nothing can take it down while the user is working:

- `run()` never raises. Besides `CodeError` it catches any unexpected exception, reports that line as an invalid code with the exception type noted, and carries on with the remaining lines.
- `evaluate_postfix()` checks the operand stack before every pop and checks that exactly one value is left at the end, so a gap in the validation would surface as a reported error rather than a crash.
- `process()` in the GUI wraps the call to the evaluator, and `load_file()` reports an unreadable file, a non-`.in` file, and a cancelled dialog without failing.

Assumptions

The specification leaves a few points open; these are the choices made here.

- **Whole numbers by default.** A value is an integer unless the code contains a decimal literal. Since `%` needs whole numbers and the variable naming rules come from C, integer `/` truncates toward zero (`-7 / 2` is `-3`) and `%` takes the sign of the left operand (`-7 % 2` is `-1`). A decimal literal switches that expression to real division; `%` on a non-integer is an invalid code.
- **Parentheses and unary signs are supported.** Neither is mentioned in the specification, but both are ordinary in infix expressions. In the postfix output a unary sign prints as `-u` or `+u` so it reads apart from the binary operator.
- **A variable counts as used** whether it is read or assigned, so a variable that only ever appeared in a failed code still shows in the list, as `undefined`.

Work distribution

Member	Part
Richelle de Arce	User interface, file loading
Cherlie Palarpalar	Input parsing, infix to postfix, postfix evaluation
Johnric Apolinario	Variable states, error handling, output formatting